

RTL 문제

정답 및 해설지

조합회로 · 순차회로 · CDC · FIFO · FSM · CPU · APB/AXI · Memory Map

구성 원칙

문제마다 새 페이지에서 시작하도록 나누고, 코드/정답/해설을 카드 형태로 정리했습니다. 코드 문항은 합성 가능한 RTL 스타일을 기준으로 작성했습니다. 단, async FIFO 는 실제 프로젝트에서 RAM primitive, CDC constraint, reset synchronization 정책에 맞게 보강해야 하는 구조 골격입니다.

[1.2.1] 4:1 Multiplexer mux_4to1

문제 조건

입력: data_in[3:0], sel[1:0] / 출력: data_out / 조건: 조합 회로

정답 코드

```
module mux_4to1 (  
    input wire [3:0] data_in,  
    input wire [1:0] sel,  
    output wire    data_out  
);  
  
    assign data_out = data_in[sel];  
  
endmodule
```

핵심 해설

- 4:1 MUX 는 4 개의 입력 중 sel 값이 가리키는 1 비트를 출력으로 내보내는 회로이다.
- sel=0 이면 data_in[0], sel=1 이면 data_in[1], sel=2 이면 data_in[2], sel=3 이면 data_in[3]이 선택된다.
- assign 만 사용했으므로 latch 나 flip-flop 이 생기지 않는 순수 조합회로이다.

[1.2.3] 3-to-8 Decoder decoder_3to8

문제 조건

입력: addr[2:0], enable / 출력: dec_out[7:0] / 조건: one-hot, enable=0 이면 모두 0

정답 코드

```
module decoder_3to8 (  
    input wire [2:0] addr,  
    input wire enable,  
    output wire [7:0] dec_out  
);  
  
    assign dec_out = enable ? (8'b0000_0001 << addr) : 8'b0000_0000;  
  
endmodule
```

핵심 해설

- Decoder 는 입력 주소 값을 출력 비트 위치로 바꾸는 회로이다.
- enable=1 이고 addr=3 이면 dec_out=8'b0000_1000 처럼 한 비트만 1 이 된다.
- enable=0 이면 어떤 addr 이 들어와도 모든 출력이 0 이다.

[1.2.5] 4-bit Magnitude Comparator compare_4bit

문제 조건

입력: a[3:0], b[3:0] unsigned / 출력: gt(a>b), eq(a==b), lt(a<b)

정답 코드

```
module compare_4bit (  
    input wire [3:0] a,  
    input wire [3:0] b,  
    output wire    gt,  
    output wire    eq,  
    output wire    lt  
);  
  
    assign gt = (a > b);  
    assign eq = (a == b);  
    assign lt = (a < b);  
  
endmodule
```

핵심 해설

- a 와 b 가 signed 로 선언되어 있지 않으므로 기본적으로 unsigned 값으로 비교한다.
- 4 비트 unsigned 의 범위는 0 부터 15 까지이다.
- 비교기는 내부적으로 큰 자리인 MSB 부터 비교하는 구조로 합성된다.

[1.2.7] 4-bit Ripple Carry Adder rca_4bit

문제 조건

입력: a[3:0], b[3:0], cin / 출력: sum[3:0], cout / 조건: full_adder 4 개 인스턴스화

정답 코드

```
module full_adder (
    input wire a,
    input wire b,
    input wire cin,
    output wire sum,
    output wire cout
);

    assign sum = a ^ b ^ cin;
    assign cout = (a & b) | (a & cin) | (b & cin);

endmodule

module rca_4bit (
    input wire [3:0] a,
    input wire [3:0] b,
    input wire cin,
    output wire [3:0] sum,
    output wire cout
);

    wire [4:0] c;

    assign c[0] = cin;
    assign cout = c[4];

    full_adder u_fa0 (.a(a[0]), .b(b[0]), .cin(c[0]), .sum(sum[0]), .cout(c[1]));
    full_adder u_fa1 (.a(a[1]), .b(b[1]), .cin(c[1]), .sum(sum[1]), .cout(c[2]));
    full_adder u_fa2 (.a(a[2]), .b(b[2]), .cin(c[2]), .sum(sum[2]), .cout(c[3]));
    full_adder u_fa3 (.a(a[3]), .b(b[3]), .cin(c[3]), .sum(sum[3]), .cout(c[4]));

endmodule
```

핵심 해설

- 각 full_adder의 sum은 현재 자리의 덧셈 결과이고, cout은 다음 자리로 넘어가는 carry이다.
- bit0에서 나온 c[1]이 bit1의 cin으로 들어가고, 이 흐름이 bit3까지 이어진다.
- 캐리가 LSB에서 MSB 방향으로 물결처럼 전달되기 때문에 Ripple Carry Adder라고 부른다.
- 마지막 c[4]가 4비트 덧셈의 최종 cout이다.

[1.2.8] Synchronous Reset D Flip-Flop dff_sync_rst

문제 조건

입력: clk, rst_n(active-low), d / 출력: q

정답 코드

```
module dff_sync_rst (  
    input wire clk,  
    input wire rst_n,  
    input wire d,  
    output reg q  
);  
  
always @(posedge clk) begin  
    if (!rst_n)  
        q <= 1'b0;  
    else  
        q <= d;  
    end  
  
endmodule
```

핵심 해설

- Synchronous reset 이므로 always 감지 리스트에는 posedge clk 만 들어간다.
- rst_n 이 0 이 되어도 즉시 q 가 0 이 되는 것이 아니라 다음 클럭 상승엣지에서 q 가 0 이 된다.
- active-low reset 이므로 리셋 조건은 if (!rst_n)으로 작성한다.

[1.2.9] Asynchronous Reset D Flip-Flop dff_async_rst

문제 조건

입력: clk, rst_n(active-low), d / 출력: q

정답 코드

```
module dff_async_rst (  
    input wire clk,  
    input wire rst_n,  
    input wire d,  
    output reg q  
);  
  
always @(posedge clk or negedge rst_n) begin  
    if (!rst_n)  
        q <= 1'b0;  
    else  
        q <= d;  
    end  
  
endmodule
```

핵심 해설

- Asynchronous reset 은 클럭과 무관하게 reset edge에 반응해야 하므로 negedge rst_n 을 감지 리스트에 넣는다.
- rst_n 이 1 에서 0 으로 떨어지는 순간 q 가 바로 0 으로 초기화된다.
- 리셋 해제 시점은 실제 회로에서 metastability 문제가 생길 수 있으므로 보통 reset deassertion synchronization 을 고려한다.

[1.2.12] 4-bit Synchronous Up Counter counter_4bit

문제 조건

입력: clk, rst_n(sync, active-low), enable / 출력: cnt[3:0] / enable=1 이면 매 클럭 +1

정답 코드

```
module counter_4bit (  
    input wire    clk,  
    input wire    rst_n,  
    input wire    enable,  
    output reg [3:0] cnt  
);  
  
always @(posedge clk) begin  
    if (!rst_n)  
        cnt <= 4'd0;  
    else if (enable)  
        cnt <= cnt + 4'd1;  
    else  
        cnt <= cnt;  
    end  
  
endmodule
```

핵심 해설

- reset 이 sync 이므로 always @(posedge clk)만 사용한다.
- enable=0 이면 cnt 값을 유지하고, enable=1 이면 1 씩 증가한다.
- 4 비트 counter 이므로 15 다음에는 자동으로 0 으로 overflow 된다.

[2.2.3] 4-bit Up-Down Counter counter_updown_4bit

문제 조건

입력: clk, rst_n(sync), enable, up_down(1: up, 0: down) / 출력: cnt[3:0]

정답 코드

```
module counter_updown_4bit (  
    input wire    clk,  
    input wire    rst_n,  
    input wire    enable,  
    input wire    up_down,  
    output reg [3:0] cnt  
);  
  
always @(posedge clk) begin  
    if (!rst_n)  
        cnt <= 4'd0;  
    else if (enable) begin  
        if (up_down)  
            cnt <= cnt + 4'd1;  
        else  
            cnt <= cnt - 4'd1;  
        end  
    end  
end  
  
endmodule
```

핵심 해설

- up_down=1 이면 up counter, up_down=0 이면 down counter 로 동작한다.
- enable=0 인 클럭에서는 cnt 가 유지된다.
- 4 비트이므로 down 상태에서 0 보다 작아지면 15 로 wrap-around 된다.

[2.2.4] Modulo-10 Counter counter_mod10

문제 조건

입력: clk, rst_n(sync), enable / 출력: cnt[3:0], tick / 조건: 0~9 반복, 9 에서 0 으로 넘어가는 클럭에 tick=1

정답 코드

```
module counter_mod10 (  
    input wire    clk,  
    input wire    rst_n,  
    input wire    enable,  
    output reg [3:0] cnt,  
    output reg    tick  
);  
  
always @(posedge clk) begin  
    if (!rst_n) begin  
        cnt <= 4'd0;  
        tick <= 1'b0;  
    end else begin  
        tick <= 1'b0;  
  
        if (enable) begin  
            if (cnt == 4'd9) begin  
                cnt <= 4'd0;  
                tick <= 1'b1;  
            end else begin  
                cnt <= cnt + 4'd1;  
            end  
        end  
    end  
end  
  
endmodule
```

핵심 해설

- Modulo-10 counter는 0 부터 9 까지만 세고 다시 0 으로 돌아간다.
- tick은 cnt 가 9 에서 0 으로 넘어가는 바로 그 클럭에만 1 이 된다.
- tick <= 1'b0 을 기본값으로 먼저 넣어두면 tick 이 여러 클럭 동안 1 로 유지되는 실수를 막을 수 있다.

[2.2.6] 8-bit SIPO Shift Register shift_sipo_8bit

문제 조건

입력: clk, rst_n(sync), serial_in / 출력: parallel_out[7:0] / 조건: LSB -> MSB 방향으로 채워짐

정답 코드

```
module shift_sipo_8bit (  
    input wire    clk,  
    input wire    rst_n,  
    input wire    serial_in,  
    output reg [7:0] parallel_out  
);  
  
always @(posedge clk) begin  
    if (!rst_n)  
        parallel_out <= 8'd0;  
    else  
        parallel_out <= {parallel_out[6:0], serial_in};  
    end  
  
endmodule
```

핵심 해설

- serial_in 은 매 클럭 LSB 위치인 parallel_out[0]으로 들어온다.
- 기존 parallel_out[0]은 parallel_out[1]로, parallel_out[1]은 parallel_out[2]로 이동한다.
- 즉, 새 데이터가 LSB 에 들어오고 기존 데이터가 MSB 방향으로 밀려가는 구조이다.

[3.1.1] Blocking 대입과 Non-blocking 대입의 본질적 차이

정답

Blocking 대입(=)은 그 줄에서 즉시 값이 갱신되어 다음 문장이 바뀐 값을 사용하고, Non-blocking 대입(<=)은 오른쪽 값을 먼저 평가한 뒤 현재 time step 끝에서 동시에 갱신된다.

핵심 해설

- 조합회로의 always @(*) 안에서는 보통 blocking 대입(=)을 사용한다.
- 클럭 기반 순차회로 always @(posedge clk) 안에서는 보통 non-blocking 대입(<=)을 사용한다.
- 순차회로에서 blocking 을 잘못 사용하면 시뮬레이션 순서 의존성이나 race condition 이 생길 수 있다.

[3.1.5] 합성 불가능한 non-synthesizable 구문 고르기

정답

일반적인 RTL 합성 기준 정답: (a), (b), (d), (f)

핵심 해설

- (a) initial begin ... end: 일반 ASIC RTL 기준으로는 합성 대상으로 보지 않는다. FPGA 초기값 용도는 도구별 예외가 있다.
- (b) #10 a = b;: 지연 시간 #10 은 시뮬레이션 시간 제어 구문이므로 합성 불가능하다.
- (d) force/release: 시뮬레이션에서 신호를 강제로 덮어쓰는 디버깅용 구문이다.
- (f) \$display: 시뮬레이션 출력용 system task 이므로 하드웨어 회로로 합성되지 않는다.
- (c) always_ff @(posedge clk)는 순차회로 작성용 SystemVerilog 구문이고, (e) assign y = a & b 는 조합회로이므로 합성 가능하다.

[3.1.4] Race condition 대표 코드 예시

정답 코드

```
always @(posedge clk) a = b;  
always @(posedge clk) b = a;
```

핵심 해설

- 두 always 블록이 같은 clock edge 에서 동시에 실행되는데, blocking 대입을 사용하고 있다.
- 시뮬레이터가 어느 always 블록을 먼저 실행하느냐에 따라 a 와 b 의 최종 값이 달라질 수 있다.
- 이런 순차회로는 non-blocking 대입을 사용해 같은 clock edge 에서 동시에 갱신되도록 작성하는 것이 안전하다.

[5.1.1] Setup time 과 Hold time 정의

정답

Setup time 은 clock edge 가 오기 전 데이터가 미리 안정되어 있어야 하는 최소 시간이고, **Hold time** 은 clock edge 가 지난 뒤에도 데이터가 계속 안정되어 있어야 하는 최소 시간이다.

핵심 해설

- setup time 위반: 데이터가 clock edge 직전에 너무 늦게 변해서 FF 가 올바르게 샘플링하지 못하는 경우이다.
- hold time 위반: clock edge 직후 데이터가 너무 빨리 변해서 FF 가 샘플링 값을 유지하지 못하는 경우이다.
- setup/hold 위반은 잘못된 값 캡처 또는 metastability 의 원인이 될 수 있다.

[5.2.7] Clock Enable D Flip-Flop dff_clken

문제 조건

입력: clk, rst_n(sync), clk_en, d / 출력: q / clk_en=1 인 클럭에만 데이터 캡처

정답 코드

```
module dff_clken (  
    input wire clk,  
    input wire rst_n,  
    input wire clk_en,  
    input wire d,  
    output reg q  
);  
  
always @(posedge clk) begin  
    if (!rst_n)  
        q <= 1'b0;  
    else if (clk_en)  
        q <= d;  
    else  
        q <= q;  
    end  
  
endmodule
```

핵심 해설

- clk_en=1 인 clock edge 에서만 d 를 q 에 저장한다.
- clk_en=0 이면 q 는 이전 값을 유지한다.
- 실제 클럭 자체를 AND 게이트로 직접 막는 방식은 glitch 문제가 생길 수 있으므로, RTL 에서는 보통 clock enable 형태를 사용한다.

[6.1.1] Synchronous Reset 장단점

정답

장점: clock 기준으로 동작하므로 reset release 가 타이밍 분석에 포함되기 쉽고, 비동기 reset 해제에 따른 metastability 위험이 작다. **단점:** clock 이 동작하지 않으면 reset 이 반영되지 않고, reset mux 가 data path 에 들어가 timing/area 부담을 줄 수 있다.

핵심 해설

- 장점 1: 리셋도 clock edge 에서만 반영되므로 일반적인 setup/hold timing 분석 흐름에 넣기 쉽다.
- 장점 2: reset deassertion 이 clock 과 무관하게 갑자기 풀리는 문제를 줄일 수 있다.
- 단점 1: clock 이 멈춰 있으면 reset 을 걸어도 FF 값이 즉시 초기화되지 않는다.
- 단점 2: 모든 FF 의 D 입력 앞에 reset 선택 논리가 추가되어 critical path 가 길어질 수 있다.

[6.1.2] Asynchronous Reset 장단점

정답

장점: clock 이 없어도 즉시 초기화할 수 있고 전원 인가/비상 초기화에 유리하다. **단점:** reset 해제 시 recovery/removal 위반과 metastability 위험이 있으며, reset tree 의 glitch 와 CDC 검증이 어려워질 수 있다.

핵심 해설

- 장점 1: clock 이 멈춘 상태에서도 reset 이 걸리면 FF 를 즉시 초기화할 수 있다.
- 장점 2: power-up 초기화나 외부 reset 버튼 처리에 직관적이다.
- 단점 1: reset 이 풀리는 시점이 clock edge 근처이면 recovery/removal timing 위반이 생길 수 있다.
- 단점 2: reset 신호가 많은 FF 에 퍼지므로 reset tree skew, glitch, domain 별 release 순서 관리가 필요하다.

[7.1.1] Metastability 발생 원리

정답

데이터가 clock edge 근처에서 변해 setup time 또는 hold time 을 위반하면 FF 내부 노드가 0 또는 1 로 즉시 결정되지 못하고 중간 전압 상태에 머무를 수 있는데, 이 불안정 상태를 metastability 라고 한다.

핵심 해설

- 정상적인 FF 는 clock edge 에서 입력 D 를 샘플링해 Q 를 0 또는 1 로 결정한다.
- 그러나 D 가 edge 바로 앞뒤로 변하면 FF 내부 latch 가 어느 값으로 수렴해야 하는지 순간적으로 결정하지 못할 수 있다.
- metastability 가 다음 로직으로 전파되면 잘못된 값, 늦은 전이, domain 간 handshake 실패가 발생할 수 있다.

[7.1.4] 2-flop Synchronizer 의 두 FF 가 속해야 하는 clock domain

정답

두 Flip-Flop 은 모두 수신측 clock domain 에 속해야 한다.

핵심 해설

- synchronizer 의 목적은 비동기 입력을 수신측 clock 기준으로 안정화하는 것이다.
- 첫 번째 FF 는 metastability 가 발생할 수 있는 지점이고, 두 번째 FF 는 그 metastability 가 안정될 시간을 한 클럭 더 제공한다.
- 따라서 두 FF 모두 clk_dst, 즉 수신측 클럭으로 동작해야 한다.

[7.1.7] CDC 가 발생하는 시나리오 고르기

정답

정답: (a), (b), (d)

핵심 해설

- (a) 100MHz domain 신호가 200MHz domain FF 에 들어가면 서로 다른 clock domain 간 전달이므로 CDC 이다.
- (b) 같은 100MHz 라도 다른 PLL 출력이면 위상 관계, skew, lock/release 조건이 다를 수 있으므로 별도 domain 으로 취급하는 것이 안전하다.
- (c) 같은 clock source 에서 분배되었고 skew 를 무시 가능하면 같은 domain 으로 볼 수 있어 일반적으로 CDC 가 아니다.
- (d) 주파수는 같아도 위상이 다르면 sampling edge 관계가 달라 setup/hold 위반 가능성이 있으므로 CDC 이다.

[7.2.1] 1-bit 2-flop Synchronizer sync_2ff

문제 조건

입력: clk_dst, rst_n(sync), sig_in(asynchronous) / 출력: sig_out

정답 코드

```
module sync_2ff (  
    input wire clk_dst,  
    input wire rst_n,  
    input wire sig_in,  
    output wire sig_out  
);  
  
    reg sync_ff1;  
    reg sync_ff2;  
  
    always @(posedge clk_dst) begin  
        if (!rst_n) begin  
            sync_ff1 <= 1'b0;  
            sync_ff2 <= 1'b0;  
        end else begin  
            sync_ff1 <= sig_in;  
            sync_ff2 <= sync_ff1;  
        end  
    end  
  
    assign sig_out = sync_ff2;  
  
endmodule
```

핵심 해설

- sig_in 은 송신측 domain 에서 온 비동기 신호이다.
- sync_ff1 에서 metastability 가 생겨도 sync_ff2 까지 가기 전 한 클럭의 안정화 시간이 생긴다.
- 단일 비트 level 신호 동기화에 적합하며, multi-bit bus 에는 단순 2FF 를 비트별로 적용하면 안 되는 경우가 많다.

[7.2.3] Edge Detector 결합 Synchronizer sync_edge_detect

문제 조건

입력: clk_dst, rst_n(sync), sig_in(asynchronous) / 출력: pulse_out / rising edge 감지

정답 코드

```
module sync_edge_detect (
    input wire clk_dst,
    input wire rst_n,
    input wire sig_in,
    output wire pulse_out
);

    reg sync_ff1;
    reg sync_ff2;
    reg sync_ff2_d;

    always @(posedge clk_dst) begin
        if (!rst_n) begin
            sync_ff1 <= 1'b0;
            sync_ff2 <= 1'b0;
            sync_ff2_d <= 1'b0;
        end else begin
            sync_ff1 <= sig_in;
            sync_ff2 <= sync_ff1;
            sync_ff2_d <= sync_ff2;
        end
    end

    assign pulse_out = sync_ff2 & ~sync_ff2_d;

endmodule
```

핵심 해설

- 먼저 sig_in 을 2FF 로 수신측 clock domain 에 동기화한다.
- 그 다음 현재 동기화 값(sync_ff2)과 한 클럭 전 값(sync_ff2_d)을 비교한다.
- 0 에서 1 로 변한 순간 sync_ff2=1, sync_ff2_d=0 이므로 pulse_out 이 1 클럭 동안 1 이 된다.

[8.2.7] Async FIFO async_fifo 구조 골격

문제 조건

파라미터: DATA_WIDTH=8, ADDR_WIDTH=4(depth=16) / write domain: clk_w / read domain: clk_r

정답 코드

```
module async_fifo #(
    parameter DATA_WIDTH = 8,
    parameter ADDR_WIDTH = 4
) (
    input wire      clk_w,
    input wire      rst_n_w,
    input wire      wr_en,
    input wire [DATA_WIDTH-1:0] wr_data,

    input wire      clk_r,
    input wire      rst_n_r,
    input wire      rd_en,
    output reg [DATA_WIDTH-1:0] rd_data,

    output wire      full,
    output wire      empty
);

    localparam DEPTH = (1 << ADDR_WIDTH);

    reg [DATA_WIDTH-1:0] mem [0:DEPTH-1];

    reg [ADDR_WIDTH:0] wbin;
    reg [ADDR_WIDTH:0] rbin;
    reg [ADDR_WIDTH:0] wgray;
    reg [ADDR_WIDTH:0] rgray;

    reg [ADDR_WIDTH:0] rgray_w1, rgray_w2;
    reg [ADDR_WIDTH:0] wgray_r1, wgray_r2;

    wire winc;
    wire rinc;
    wire [ADDR_WIDTH:0] wbin_next;
    wire [ADDR_WIDTH:0] rbin_next;
    wire [ADDR_WIDTH:0] wgray_next;
    wire [ADDR_WIDTH:0] rgray_next;

    assign winc = wr_en & ~full;
    assign rinc = rd_en & ~empty;

    assign wbin_next = wbin + winc;
    assign rbin_next = rbin + rinc;
    assign wgray_next = bin2gray(wbin_next);
    assign rgray_next = bin2gray(rbin_next);

    function [ADDR_WIDTH:0] bin2gray;
        input [ADDR_WIDTH:0] bin;
        begin
            bin2gray = (bin >> 1) ^ bin;
        end
    endfunction

    // Write pointer and memory write
    always @(posedge clk_w) begin
```

```

if (!rst_n_w) begin
    wbin <= {ADDR_WIDTH+1{1'b0}};
    wgray <= {ADDR_WIDTH+1{1'b0}};
end else begin
    if (winc)
        mem[wbin[ADDR_WIDTH-1:0]] <= wr_data;
    wbin <= wbin_next;
    wgray <= wgray_next;
end
end

// Read pointer and memory read
always @(posedge clk_r) begin
    if (!rst_n_r) begin
        rbin <= {ADDR_WIDTH+1{1'b0}};
        rgray <= {ADDR_WIDTH+1{1'b0}};
        rd_data <= {DATA_WIDTH{1'b0}};
    end else begin
        if (rinc)
            rd_data <= mem[rbin[ADDR_WIDTH-1:0]];
        rbin <= rbin_next;
        rgray <= rgray_next;
    end
end

// Read gray pointer synchronized into write clock domain
always @(posedge clk_w) begin
    if (!rst_n_w) begin
        rgray_w1 <= {ADDR_WIDTH+1{1'b0}};
        rgray_w2 <= {ADDR_WIDTH+1{1'b0}};
    end else begin
        rgray_w1 <= rgray;
        rgray_w2 <= rgray_w1;
    end
end

// Write gray pointer synchronized into read clock domain
always @(posedge clk_r) begin
    if (!rst_n_r) begin
        wgray_r1 <= {ADDR_WIDTH+1{1'b0}};
        wgray_r2 <= {ADDR_WIDTH+1{1'b0}};
    end else begin
        wgray_r1 <= wgray;
        wgray_r2 <= wgray_r1;
    end
end

assign empty = (rgray_next == wgray_r2);

assign full = (wgray_next == {~rgray_w2[ADDR_WIDTH:ADDR_WIDTH-1],
    rgray_w2[ADDR_WIDTH-2:0]});

endmodule

```

핵심 해설

- write domain 에서는 write binary pointer 와 write gray pointer 를 가진다.
- read domain 에서는 read binary pointer 와 read gray pointer 를 가진다.
- 주소 계산은 binary pointer 로 하고, clock domain 을 넘어갈 때는 gray pointer 를 동기화한다.
- gray code 는 한 번에 1 비트만 바뀌므로 multi-bit pointer 를 CDC 로 넘길 때 binary 보다 안전하다.
- empty 는 read domain 에서 동기화된 write pointer 와 read pointer 를 비교해 판단한다.
- full 은 write domain 에서 동기화된 read pointer 와 write pointer 를 비교해 판단한다.
- 실제 제품용 FIFO 에서는 RAM inference 방식, false path/CDC constraint, reset release 정책, almost_full/almost_empty 등을 추가 검토해야 한다.

[11.1.1] Moore FSM 과 Mealy FSM 의 가장 큰 차이

정답

Moore FSM 의 출력은 현재 state 만으로 결정되고, Mealy FSM 의 출력은 현재 state 와 현재 input 을 함께 사용해 결정된다.

핵심 해설

- Moore: $\text{output} = f(\text{state})$
- Mealy: $\text{output} = f(\text{state}, \text{input})$
- 출력이 input 에 즉시 반응하는지 여부가 핵심 차이이다.

[11.1.2] Moore 와 Mealy 의 장단점

정답

Mealy FSM 은 입력 변화에 바로 반응할 수 있어 latency 가 짧고 상태 수가 줄어 들 수 있다. Moore FSM 은 출력이 state register 에 의해 안정적으로 결정되므로 glitch 에 더 강하지만 보통 1 클럭 늦게 반응할 수 있다.

핵심 해설

- Moore 장점: 출력이 state 에만 의존하므로 비교적 안정적이고 glitch 에 강하다.
- Moore 단점: 입력 이벤트에 대한 출력 반응이 다음 state 이후에 나타나 latency 가 길어질 수 있다.
- Mealy 장점: 입력 변화가 출력에 바로 반영될 수 있어 latency 가 짧다.
- Mealy 단점: 입력 glitch 가 출력 glitch 로 이어질 수 있어 timing 과 glitch 관리가 더 중요하다.

[11.2.1] 3-state Moore FSM fsm_moore_3state - 1-process 스타일

문제 조건

상태: IDLE -> READY -> DONE -> IDLE / 입력: clk, rst_n(sync), start, done_in / 출력: state[1:0], busy, complete

정답 코드

```
module fsm_moore_3state (
    input wire    clk,
    input wire    rst_n,
    input wire    start,
    input wire    done_in,
    output reg [1:0] state,
    output reg    busy,
    output reg    complete
);

localparam IDLE = 2'd0;
localparam READY = 2'd1;
localparam DONE = 2'd2;

always @(posedge clk) begin
    if (!rst_n) begin
        state <= IDLE;
        busy <= 1'b0;
        complete <= 1'b0;
    end else begin
        case (state)
            IDLE: begin
                busy <= 1'b0;
                complete <= 1'b0;
                if (start)
                    state <= READY;
                else
                    state <= IDLE;
            end

            READY: begin
                busy <= 1'b1;
                complete <= 1'b0;
                if (done_in)
                    state <= DONE;
                else
                    state <= READY;
            end

            DONE: begin
                busy <= 1'b0;
                complete <= 1'b1;
                state <= IDLE;
            end

            default: begin
                state <= IDLE;
                busy <= 1'b0;
                complete <= 1'b0;
            end
        endcase
    end
end
```



```
    endcase  
  end  
end  
  
endmodule
```

핵심 해설

- 1-process 스타일은 state register, state transition, output register 를 하나의 clocked always 블록 안에 함께 작성한다.
- Moore FSM 이므로 busy 와 complete 는 현재 state 에 대응되는 값으로 출력된다.
- DONE 상태는 complete=1 을 한 클럭 출력한 뒤 다음 클럭에 IDLE 로 돌아간다.

11. FSM

[11.2.2] 3-state Moore FSM fsm_moore_3state_3p - 3-process 스타일

정답 코드

```
module fsm_moore_3state_3p (  
    input wire    clk,  
    input wire    rst_n,  
    input wire    start,  
    input wire    done_in,  
    output reg [1:0] state,  
    output reg    busy,  
    output reg    complete  
);  
  
localparam IDLE = 2'd0;  
localparam READY = 2'd1;  
localparam DONE = 2'd2;  
  
reg [1:0] next_state;  
  
// 1) State register  
always @(posedge clk) begin  
    if (!rst_n)  
        state <= IDLE;  
    else  
        state <= next_state;  
end  
  
// 2) Next state logic  
always @(*) begin  
    next_state = state;  
  
    case (state)  
        IDLE: begin  
            if (start)  
                next_state = READY;  
            end  
  
        READY: begin  
            if (done_in)  
                next_state = DONE;  
            end  
  
        DONE: begin  
            next_state = IDLE;  
            end  
  
        default: begin  
            next_state = IDLE;  
            end  
    endcase  
end  
  
// 3) Output logic  
always @(*) begin  
    busy    = 1'b0;  
    complete = 1'b0;  
  
    case (state)  
        IDLE: begin  
            busy    = 1'b0;  
            complete = 1'b0;  
        end  
    end  
end
```

```
READY: begin
    busy  = 1'b1;
    complete = 1'b0;
end

DONE: begin
    busy  = 1'b0;
    complete = 1'b1;
end

default: begin
    busy  = 1'b0;
    complete = 1'b0;
end
endcase
end

endmodule
```

핵심 해설

- 3-process 스타일은 state register, next state logic, output logic 을 분리한다.
- next_state logic 은 조합회로이므로 항상 기본값을 먼저 주어 latch 생성을 막는다.
- output logic 도 조합회로이며 Moore FSM 이므로 state 만 보고 busy/complete 를 결정한다.

[14.1.3] Multi-cycle CPU 와 Single-cycle CPU 차이

정답

Single-cycle CPU 는 모든 명령어를 한 클럭에 완료하므로 clock period 가 가장 느린 명령어에 맞춰 길어져야 한다. Multi-cycle CPU 는 명령어 실행을 여러 단계로 나누고 명령어 종류에 따라 필요한 cycle 수만 사용하므로 하드웨어 재사용과 clock period 단축이 가능하다.

핵심 해설

- Single-cycle 은 IF, ID, EX, MEM, WB 에 해당하는 동작이 한 클럭 안에 끝나야 한다.
- Multi-cycle 은 instruction fetch, decode, execute, memory access, write-back 등을 여러 클럭에 나눠 수행한다.
- multi-cycle 에서 더 많은 cycle 을 쓰는 대표 명령어는 load 계열이다. 주소 계산, data memory read, register write-back 이 필요하기 때문이다.
- store 는 memory write 는 필요하지만 register write-back 이 없어서 load 보다 보통 한 단계 적을 수 있다. R-type ALU 명령은 memory access 가 없어 상대적으로 적은 cycle 을 사용한다.

[16.1.1] APB 주요 신호 8 가지 역할

정답

PSEL 은 slave 선택, PENABLE 은 access phase 표시, PWRITE 는 write/read 방향, PADDR 은 주소, PWDATA 는 write data, PRDATA 는 read data, PREADY 는 slave ready/wait, PSLVERR 는 transfer error 를 나타낸다.

핵심 해설

- PSEL: 해당 APB slave 가 선택되었음을 나타낸다.
- PENABLE: setup phase 다음 access phase 임을 나타낸다.
- PWRITE: 1 이면 write transfer, 0 이면 read transfer 이다.
- PADDR: 접근할 register 또는 memory 주소이다.
- PWDATA: master 가 slave 에 쓰는 데이터이다.
- PRDATA: slave 가 master 에 돌려주는 읽기 데이터이다.
- PREADY: slave 가 transfer 를 완료할 준비가 되었는지 나타낸다.
- PSLVERR: transfer 가 에러로 끝났는지 나타낸다.

[16.1.3] APB Slave wait state 생성 신호와 동작

정답

APB slave 가 wait state 를 만들 때 사용하는 신호는 PREADY 이다. Access phase 에서 PREADY=0 으로 유지하면 master 는 PSEL, PENABLE, PADDR, PWRITE, PWDATA 등을 유지하며 기다리고, PREADY=1 이 되는 클럭에 transfer 가 완료된다.

핵심 해설

- APB transfer 는 setup phase 와 access phase 로 나뉜다.
- access phase 에서 PSEL=1, PENABLE=1 인 상태가 된다.
- 이때 PREADY=0 이면 wait state 가 삽입된다.
- PREADY=1 이면서 PSEL=1, PENABLE=1 인 클럭에서 transfer 가 완료된다.

[17.1.1] AXI4 5 개 채널 이름과 운반 정보

정답

AW 는 write address, W 는 write data, B 는 write response, AR 은 read address, R 은 read data/response 를 운반한다.

핵심 해설

- AW 채널: write address 와 burst/control 정보를 전달한다.
- W 채널: write data, byte strobe, 마지막 beat 여부 등을 전달한다.
- B 채널: write transaction 의 응답 상태를 slave 가 master 에게 전달한다.
- AR 채널: read address 와 burst/control 정보를 전달한다.
- R 채널: read data, read response, 마지막 beat 여부 등을 전달한다.

17. AXI4

[17.1.2] AXI4 공통 핸드셰이크 신호와 데이터 전송 조건

정답

AXI4 의 모든 채널은 VALID 와 READY 를 공통 핸드셰이크로 사용한다. 해당 채널에서 VALID=1 이고 READY=1 인 clock edge 에 transfer 가 성립한다.

핵심 해설

- VALID 는 보내는 쪽이 유효한 정보 또는 데이터를 내놓았다는 뜻이다.
- READY 는 받는 쪽이 그 정보를 받을 준비가 되었다는 뜻이다.
- 둘 중 하나라도 0 이면 transfer 는 일어나지 않고, source 는 보통 VALID 와 payload 를 유지해야 한다.
- AVALID/AWREADY, WVALID/WREADY, BVALID/BREADY, ARVALID/ARREADY, RVALID/RREADY 처럼 채널마다 이름이 붙는다.

18. Memory Map 과 Address Decoding

[18.1.1] Memory Map 정의와 SoC 에서 필요한 이유

정답

Memory map 은 SoC 의 전체 주소 공간 중 어느 주소 범위가 어떤 memory, peripheral, register block 에 연결되는지를 정한 주소 배치표이다. CPU 와 bus interconnect 가 주소를 보고 올바른 slave 로 접근을 보내기 위해 필요하다.

핵심 해설

- 소프트웨어 입장에서는 특정 주소에 read/write 하면 특정 하드웨어 register 를 제어할 수 있어야 한다.
- 하드웨어 입장에서는 master 가 낸 주소를 보고 어느 slave 를 선택할지 결정해야 한다.
- 따라서 memory map 은 소프트웨어 드라이버와 하드웨어 address decoder 사이의 약속이다.

[18.1.2] Address Decoding 기본 동작 원리

정답

Address decoding 은 입력 주소가 어떤 base address/range 에 속하는지 비교해서 해당 slave select 신호를 1 로 만드는 회로이다.

핵심 해설

- 예를 들어 0x4000_0000~0x4000_0FFF 는 GPIO, 0x4000_1000~0x4000_1FFF 는 UART 처럼 범위를 나눈다.
- decoder 는 주소의 상위 비트 또는 base/mask 비교 결과로 PSEL, chip select, enable 같은 slave select 신호를 만든다.
- 보통 하나의 주소는 하나의 slave 만 선택되도록 one-hot 형태로 설계한다.

[18.1.3] Base Address 와 Offset 의미

정답

Base address 는 한 slave/register block 이 시작되는 기준 주소이고, offset 은 그 base address 로부터 떨어진 내부 위치이다. 한 slave 내부의 여러 register 는 base address + offset 형태의 주소로 구분된다.

핵심 해설

- 예를 들어 GPIO base 가 0x4000_0000 이고 DATA register offset 이 0x00 이면 DATA 주소는 0x4000_0000 이다.
- DIR register offset 이 0x04 이면 DIR 주소는 0x4000_0004 이다.
- 하드웨어 내부에서는 보통 PADDR - BASE 또는 PADDR 의 하위 비트를 사용해 어떤 register 를 접근할지 선택한다.

제출/검증 팁

- 조합회로 always @(*)에서는 모든 출력에 기본값을 주어 latch 생성을 막는다.
- 순차회로 always @(posedge clk)에서는 non-blocking 대입(<=)을 기본으로 사용한다.
- 동기 reset 은 감지 리스트에 reset edge 를 넣지 않고, 비동기 reset 은 reset edge 를 감지 리스트에 넣는다.
- CDC 에서는 단일 bit level 신호는 2FF synchronizer, pulse 는 toggle/handshake, multi-bit data 는 FIFO 또는 handshake 기반 전달을 우선 고려한다.
- AXI/APB 같은 bus protocol 은 신호 이름보다 transfer 성립 조건을 먼저 잡으면 이해가 쉽다.